Fundamentos de PyBullet

Análisis Arquitectónico del Entorno de Simulación

Clase de Robótica

April 2, 2026

¿Por qué necesitamos conocer la posición?

Antes de programar, pensemos: **¿Para qué nos sirve saber dónde está el robot?**
En robótica, a esto se le llama **Odometría** o **Telemetría**. Es el pilar de la autonomía por tres razones principales:

- **Control de Lazo Cerrado:** Si le decimos al robot "Avanza 1 metro", los motores no son perfectos. Las ruedas patinan, la fricción cambia y el peso afecta. Leer la posición real nos permite medir y corregir ese error.
- **Navegación:** Para ir del Punto A al Punto B, el robot necesita responder continuamente a la pregunta: *"¿Dónde estoy ahora mismo en relación a mi destino?"*.
- **Mapeo y Evasión:** Si los sensores detectan un obstáculo, necesitamos saber nuestra posición exacta en el mundo para marcar ese obstáculo correctamente en nuestro mapa virtual.

1. Obteniendo la posición del Robot

En PyBullet, necesitamos saber dónde está nuestro robot en el mundo tridimensional. Para esto utilizamos la función `getBasePositionAndOrientation`.

```
posicion, orientacion = p.getBasePositionAndOrientation(
    robot_id)
```

¿Qué devuelve esta función?

- `posicion`: Una tupla con 3 valores (X, Y, Z) en metros.
- `orientacion`: Una tupla con 4 valores representando un cuaternión (X, Y, Z, W).

Extrayendo X e Y

Como nuestro robot se mueve en el suelo, el eje Z (altura) no nos interesa tanto para la navegación. Separamos los datos así:

```
x, y, z = posicion
print("El robot esta en X:", x, " e Y:", y)
```

En Python, podemos "desempaquetar" una tupla directamente asignando variables separadas por comas.

¿Cómo definimos hacia dónde mira el robot?

Sabemos que la **posición** se define fácilmente con tres coordenadas Cartesianas (X, Y, Z).
Pero la posición no nos dice hacia dónde está apuntando el "frente" de nuestro robot. Para la navegación, necesitamos conocer su **orientación** o rotación en el espacio.

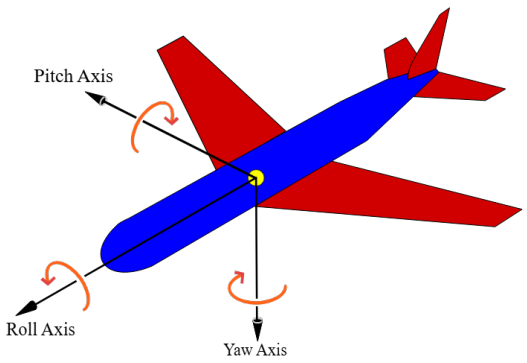
El desafío matemático

Representar rotaciones en 3D de forma computacional es mucho más complejo que representar posiciones. Existen diferentes formas de hacerlo, cada una con sus ventajas y desventajas.

1. Ángulos de Euler (La forma intuitiva)

La forma más natural para los humanos de entender la rotación es usando los **Ángulos de Euler**. Dividimos la rotación en tres giros independientes alrededor de los ejes principales:

- **Roll (Alabeo):** Rotación en el eje X. (Imagina un avión inclinando las alas).
- **Pitch (Cabeceo):** Rotación en el eje Y. (Inclinarse hacia arriba o abajo).
- **Yaw (Guiñada):** Rotación en el eje Z. (Girar a la izquierda o derecha).



Ejes Roll, Pitch, Yaw

Para nuestro robot de 4 ruedas en el suelo, el **Yaw** *(θ) es el único ángulo que cambiaremos activamente.*

El problema de Euler: "Gimbal Lock"

Si Euler es tan fácil de entender, **¿Por qué PyBullet no lo usa por defecto?**
El problema ocurre por un fenómeno matemático y físico llamado **Gimbal Lock** (Bloqueo del Cardán).

- Las rotaciones de Euler se aplican en un orden específico (ej. primero X, luego Y, luego Z).
- Si rotamos un eje exactamente 90° (como mirar directamente hacia arriba), **dos ejes se alinean**.
- Al alinearse, ¡perdemos un grado de libertad! El sistema colapsa matemáticamente (singularidad) y el simulador no sabría cómo calcular el siguiente movimiento.

2. Cuaterniones (La solución computacional)

Para evitar que el motor de físicas de PyBullet colapse, se utiliza una herramienta matemática llamada **Cuaterniones** (Quaternions).

¿Qué son? Son números hipercomplejos de 4 dimensiones. Se representan con una tupla de 4 valores:

$$q = (X, Y, Z, W)$$

Ventajas:

- **NO sufren de Gimbal Lock.**
- Son extremadamente eficientes de calcular para el procesador.
- Las transiciones y giros son suaves.

El inconveniente

¡Son casi imposibles de visualizar e interpretar por un ser humano! Por eso, en nuestros scripts **leemos cuaterniones, pero los convertimos a Euler** para tomar decisiones.

2. Cuaterniones (La solución computacional)

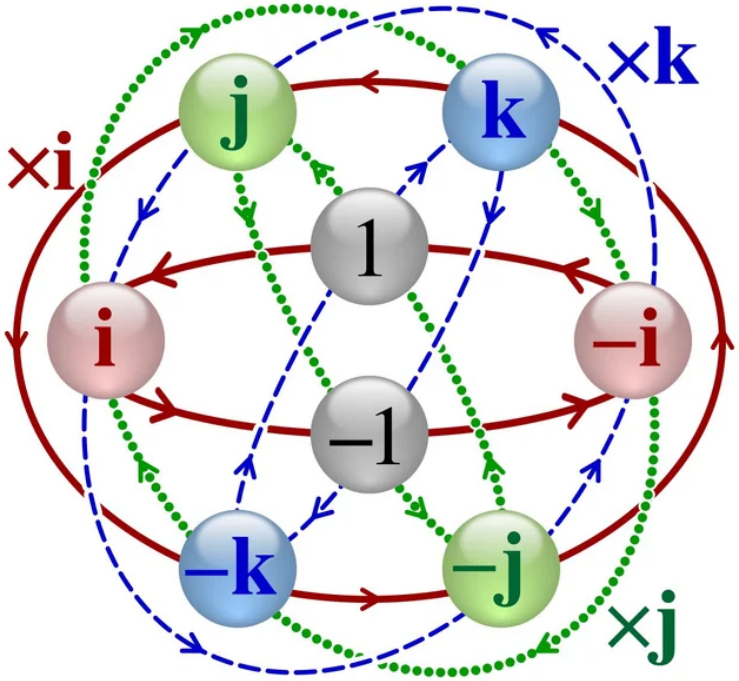


Figure: Cuaterniones (La solución computacional)

Entendiendo la Orientación (Cuaterniones a Euler)

Los cuaterniones evitan el "Gimbal Lock", pero son difíciles de leer para los humanos. Los convertiremos a Ángulos de Euler (Roll, Pitch, Yaw).

```
angulos_euler = p.getEulerFromQuaternion(orientacion)
roll, pitch, yaw = angulos_euler
```

- **Roll:** Inclinación lateral (eje X).
- **Pitch:** Inclinación frontal (eje Y).
- **Yaw (θ):** Rotación sobre su propio centro (eje Z). **¡Este es el que define hacia dónde mira el robot!**

2. Moviendo el Robot: La Función Principal

Para que nuestro robot se mueva en el simulador, debemos enviar comandos a los "joints" (articulaciones) que definimos en el URDF. En PyBullet, la función clave para esto es:

```
p.setJointMotorControl2(
    bodyUniqueId=robot_id,
    jointIndex=numero_articulacion,
    controlMode=p.VELOCITY_CONTROL,
    targetVelocity=velocidad,
    force=fuerza_maxima
)
```

Analizaremos cada parámetro en detalle, ya que un error aquí hará que el robot no se mueva o se comporte de manera errática.

Anatomía del Comando: Identificando el Objetivo

Los dos primeros parámetros le dicen al simulador **qué motor exacto** queremos mover en todo el universo virtual.

- `bodyUniqueId` (`robot_id`): Es el número identificador del robot completo. Lo obtenemos al momento de cargar el URDF con `p.loadURDF()`. Si tuviéramos 5 robots en escena, esto evita que movamos la rueda del robot equivocado.
- `jointIndex` (`numero_articulacion`): Es el número entero (0, 1, 2, etc.) que identifica la articulación específica dentro de ese robot. *(Veremos cómo saber qué número es cada rueda en la siguiente diapositiva).*

Anatomía del Comando: Modos de Control

El parámetro `controlMode` define *cómo* queremos que actúe el motor. PyBullet ofrece varios, pero los principales para robótica móvil son:

- `p.POSITION_CONTROL`: El motor intenta llegar a un ángulo específico y detenerse. *Ideal para brazos robóticos o servos de dirección.*
- `p.TORQUE_CONTROL`: Aplicamos una fuerza bruta constante, independiente de la velocidad.
- `p.VELOCITY_CONTROL`: El motor gira infinitamente intentando mantener una velocidad constante. **¡Este es el que usamos para nuestras ruedas!**

Anatomía del Comando: Velocidad y Fuerza

Una vez establecido el modo de velocidad, debemos definir la magnitud del movimiento:

- `targetVelocity` (`velocidad`): Es la velocidad angular deseada, medida en **radianes por segundo (rad/s)**.
- `force` (`fuerza_maxima`): Es el torque máximo (Newton-metro) que el motor puede ejercer para intentar alcanzar esa velocidad.

Consideración Física

Nuestro robot modelado tiene un chasis de 10kg y ruedas de 1kg cada una. Si le pedimos una `targetVelocity` alta pero la `force` por defecto es muy baja, ¡el robot no tendrá la fuerza suficiente para vencer su propia inercia y no se moverá!

¿De dónde salen los índices de las ruedas?

Para el parámetro `jointIndex`, PyBullet asigna los números **en el mismo orden en que escribimos las etiquetas** `<joint>` **en nuestro archivo XML (URDF)**.

Revisando nuestro `robot_movil.urdf`:

0. `joint_rueda_dd` (Delantera Derecha)
1. `joint_rueda_di` (Delantera Izquierda)
2. `joint_rueda_td` (Trasera Derecha)
3. `joint_rueda_ti` (Trasera Izquierda)

```
ruedas_derechas = [0, 2] # dd y td
ruedas_izquierdas = [1, 3] # di y ti
```

Buenas Prácticas: Descubriendo Joints por Código

Si bajamos un URDF de internet y no sabemos el orden, podemos usar código para inspeccionarlo en lugar de leer el XML a mano:

```
num_joints = p.getNumJoints(robot_id)

for i in range(num_joints):
    # Obtiene info detallada de la articulacion
    info = p.getJointInfo(robot_id, i)
    nombre_articulacion = info[1].decode('utf-8')
    print(f"Indice {i}: {nombre_articulacion}")
```

Esto imprimirá en consola la lista exacta de índices y sus nombres, facilitando la asignación de variables.

3. La Estrategia de Navegación

Queremos que el robot vaya desde su posición actual (x, y) hasta un punto objetivo (X_d, Y_d).
Para lograrlo con un chasis diferencial (como el nuestro), debemos dividir el problema en dos tareas simultáneas:

- **Traslación:** Avanzar hacia adelante para reducir la distancia.
- **Rotación:** Girar sobre su propio eje para apuntar hacia el objetivo.

El Concepto del Error

En la teoría de control, el "Error" no significa "equivocación". Significa **la diferencia matemática entre dónde estoy y dónde quiero estar**.
Nuestro objetivo es hacer que el error llegue a cero.

Paso 1: Calculando el Error de Distancia

¿Cuánto nos falta para llegar al objetivo? Calculamos la distancia en línea recta usando el Teorema de Pitágoras.

```
error_distancia = math.sqrt((X_d - x)**2 + (Y_d - y)**2)
```

$$E_d = \sqrt{(X_d - x)^2 + (Y_d - y)^2}$$

- X_d, Y_d: Coordenadas del destino (Target).
- x, y: Coordenadas actuales del robot.
- E_d: Siempre será un número positivo (en metros). Cuando E_d sea muy cercano a 0, sabremos que hemos llegado.

Paso 2: Calculando el Error de Ángulo

¿Hacia dónde deberíamos mirar? Primero calculamos el ángulo absoluto hacia la meta y luego lo comparamos con el ángulo hacia el que mira el robot (*Yaw*).

```
angulo_deseado = math.atan2(Y_d - y, X_d - x)
error_angulo = angulo_deseado - yaw_actual
```

¿Por qué usar atan2 y no atan?

La función matemática tangente inversa (`atan`) no sabe distinguir en qué cuadrante estamos (ej. no distingue entre ir hacia adelante-derecha o atrás-izquierda). `atan2(y, x)` toma ambos valores por separado y nos da el ángulo exacto considerando los 4 cuadrantes del plano cartesiano.

Paso 3: Controlador Proporcional (La "P" del PID)

Ya sabemos cuánto nos falta (Errores). Ahora debemos decidir **qué tan rápido movernos**.
Usaremos un Control Proporcional: la velocidad será directamente proporcional al tamaño del error.

```
Kp_lineal = 2.0
Kp_angular = 5.0

vel_lineal = Kp_lineal * error_distancia
vel_angular = Kp_angular * error_angulo
```

La intuición física: Si el robot está muy lejos de la meta, E_d es grande y acelerará a fondo. A medida que se acerca, E_d se reduce y el robot **frenará suavemente** hasta detenerse en la meta. Las constantes K_p son los "multiplicadores de agresividad".

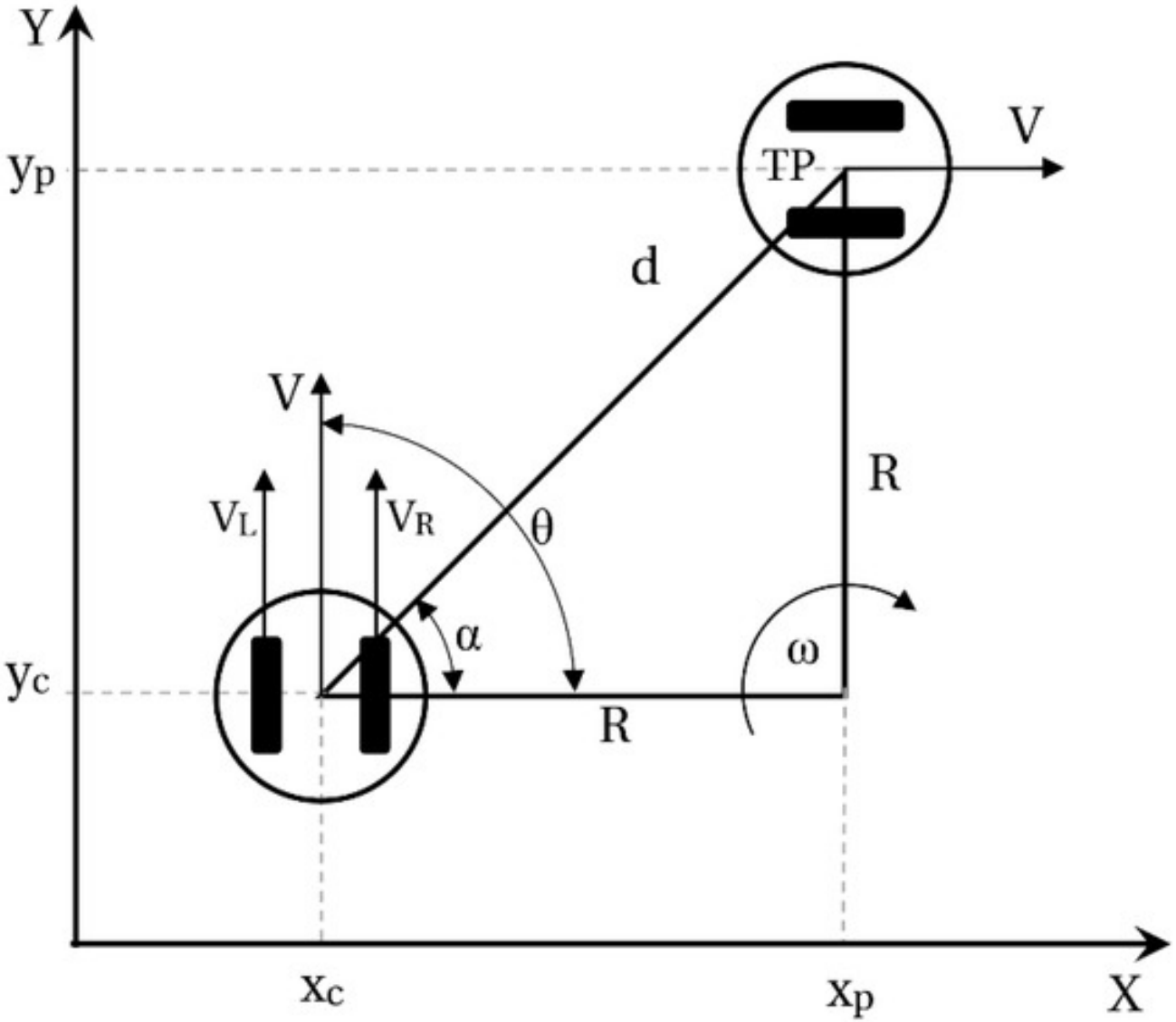
Paso 4: La Mezcla Diferencial (Skid-Steer)

Tenemos una `vel_lineal` conceptual y una `vel_angular` conceptual. Pero nuestro robot solo entiende de velocidades para ruedas derechas e izquierdas.

¿Cómo transformamos los conceptos en comandos físicos?

```
vel_izq = vel_lineal - vel_angular
vel_der = vel_lineal + vel_angular
```

Análisis de casos:

- Si solo queremos avanzar (`vel_angular = 0`): Ambas ruedas giran a la misma velocidad (`vel_lineal`).
- Si queremos girar a la izquierda (`vel_angular > 0`): La rueda derecha suma velocidad y la izquierda resta. ¡El robot pivota!

Control Proporcional